

embed-api

A production-minded embedding API for **multilingual-e5-large**

FastAPI · sentence-transformers · CPU · Docker · GitHub Actions · k3s

The model shapes the API

e5 fact	What the API does about it
Trained with <code>query:</code> / <code>passage:</code> prefixes; quality drops without them	<code>input_type</code> is required ; the server adds the prefix
Reads at most 512 tokens , prefix and special tokens included	Truncates and reports <code>tokens</code> + <code>truncated</code> per input
~560M params, CPU-bound, ~0.1–1 s per request	Inference off the event loop, one batch at a time, token budget per request
Output is L2-normalised, 1024-dim	Documented: cosine similarity = dot product
Hub repo is 9.5 GB, <code>main</code> moves	Pinned revision, download an allow-list, load by path, offline at runtime

What the brief asked for → decisions

- **Input validation:** pydantic schema = validation = Swagger docs. Empty/blank text, ≤ 64 inputs, ≤ 8000 chars, no unknown fields. Body-size middleware *before* parsing; token budget *after* tokenising.
- **Error handling:** one envelope `{error: {code, message, request_id}}`; stable codes (`model_not_ready`, `token_budget_exceeded`, ...); 500s hide internals.
- **Logging:** JSON, one access line per request: latency, inputs, tokens, truncations. `X-Request-ID` on every response, log line and error. **Input text is never logged.**
- **Good practice:** `Embedder` protocol → 31 fast tests with a fake model in < 1 s, plus slow tests on the real model. Ruff, CI, typed settings via `EMBED_*` env vars.

Request lifecycle

```
client → RequestContext (request id, access log)
      → BodySizeLimit (413 before parsing)
      → pydantic validation (422)
      → readiness gate (503 while the model loads)
      → tokenise in a worker thread → token budget (422)
      → semaphore → encode in a worker thread → response
```

- Model loads **in the background**: liveness passes at once, readiness flips when warm
- Torch thread count follows the **container CPU limit**, not the host's cores

Faster inference: measure speed *and* quality

Backend	Weights	p50, 1 text	p50, 8 texts	p50, 32 texts	Texts/s	Cosine vs fp32	Same top-1
PyTorch fp32	2,240 MB	114 ms	494 ms	1,842 ms	17	1.0000	100%
ONNX fp32	2,236 MB	58 ms	339 ms	1,335 ms	24	1.0000	100%
ONNX int8 (AVX2)	562 MB	36 ms	233 ms	931 ms	34	0.9948	100%

- int8 dynamic quantisation, exported for **AVX2** (the deployment CPU); the Hub's int8 file needs AVX-512 VNNI
- 4 threads (the pod's CPU limit), ~25-token inputs
- Fidelity vs fp32 on a small Danish/English retrieval set: **2× throughput**, **¼ the size**, **same rankings**
- → **int8 is the default**. Quantised at build time; runtime is ONNX Runtime + tokenizers, **no torch** (image 1.8 → 0.6 GB compressed). A test pins int8 ≈ fp32.

Shipping it

- **Docker:** multi-stage; int8 model quantised in a build stage from a pinned revision; no torch at runtime; non-root, read-only root fs
- **CI** (GitHub Actions): ruff + tests → build image → **smoke-test the container** → push to GHCR on `main`
- **CD:** GitOps on my k3s homelab: ArgoCD + image updater roll out each new digest; probes on `/health/live` and `/health/ready`

What I'd add for real production: auth + rate limiting · Prometheus metrics · queue timeout with `Retry-After` · cross-request dynamic batching · GPU · or Hugging Face TEI instead of a hand-written server

Demo

1. Swagger at `/docs`
2. A validation error, and its request id in the logs
3. `examples/demo.py` : Danish, English and German side by side